

À quoi sert ce fichier ?

Ce fichier centralise **toutes les adresses (URLs)** du backend. Au lieu d'écrire l'adresse du serveur dans chaque page (et risquer une faute de frappe ou un oubli si elle change), on l'écrit une seule fois ici.

Le code, expliqué

```
class ApiConfig {  
  static const String baseUrl =  
    'https://agriscan-backend-04bd.onrender.com';
```

- `baseUrl` est l'adresse de base de notre serveur (hébergé sur Render).
- `static const` veut dire : cette valeur est fixe et accessible directement via `ApiConfig.baseUrl`, sans avoir besoin de créer un objet `ApiConfig()`.

```
static const String login = '$baseUrl/api/auth/login';
```

- On colle `baseUrl` avec le chemin spécifique (`/api/auth/login`) grâce au symbole `$`.
- Résultat final : `https://agriscan-backend-04bd.onrender.com/api/auth/login`

Les URLs fixes (toujours les mêmes)

```
static const String login = '$baseUrl/api/auth/login';  
static const String register = '$baseUrl/api/auth/register';  
static const String forgotPassword = '$baseUrl/api/auth/forgot-password';  
static const String resetPassword = '$baseUrl/api/auth/reset-password';  
static const String changePassword = '$baseUrl/api/auth/change-password';  
static const String health = '$baseUrl/actuator/health';
```

Chacune correspond à une action précise sur le backend (se connecter, s'inscrire, réinitialiser le mot de passe, etc.).

Les URLs dynamiques (qui changent selon un ID)

```
static String client(int id) => '$baseUrl/api/clients/$id';
```

- Ici, ce n'est pas une simple variable, mais une **fonction**.
- Elle prend un `id` en paramètre (par exemple `42`)
- Elle retourne l'URL complète avec cet ID dedans : `https://.../api/clients/42`

C'est utile car chaque utilisateur a un ID différent — on ne peut pas écrire l'URL à l'avance.

```
static String uploadPhoto(int id) => '$baseUrl/api/clients/$id/photo';
```

```
static String historique(int clientId) =>  
  '$baseUrl/api/conversations/client/$clientId';  
static String conversation(int id) => '$baseUrl/api/conversations/$id';  
static String ajouterMessage(int id) =>  
  '$baseUrl/api/conversations/$id/messages';
```

Même principe pour : uploader une photo, récupérer l'historique d'un client, accéder à une conversation précise, ou ajouter un message dedans.

Comment ce fichier est utilisé ailleurs

Dans n'importe quelle page, par exemple `login_page.dart` :

```
final response = await http.post(
  Uri.parse(ApiConfig.login), // ← on réutilise l'URL définie ici
  ...
);
```

Pourquoi c'est une bonne pratique

Imagine que l'adresse du serveur change (nouveau nom de domaine). Sans ce fichier, il faudrait modifier l'URL dans **chaque** page une par une. Avec `ApiConfig`, on change **une seule ligne** (`baseUrl`) et tout le reste de l'app suit automatiquement.

Lexique

Terme	Explication
<code>static</code>	La valeur/fonction appartient à la classe elle-même, pas à une instance. On y accède directement avec <code>ApiConfig.xxx</code>
<code>const</code>	La valeur ne change jamais, elle est fixée une fois pour toutes
Interpolation de chaîne (<code>\$variable</code>)	Permet d'insérer la valeur d'une variable directement dans un texte

1. Footnote details# Module 01 — `services/api_config.dart` <#>